



Chapter 1: Introduction

This chapter will give you a whole picture of database (DB) management system and design of databases. It will introduce some important concepts and fundamentals about DB. These concepts and fundamentals will be explained in detail in the following chapters

Database Principle and Design



Outline

- Database Applications
- Purpose of Database Systems
- View of Data
- Database Languages
- Database Design
- Database System Structure
- Database Application Architecture
- History of Database Systems



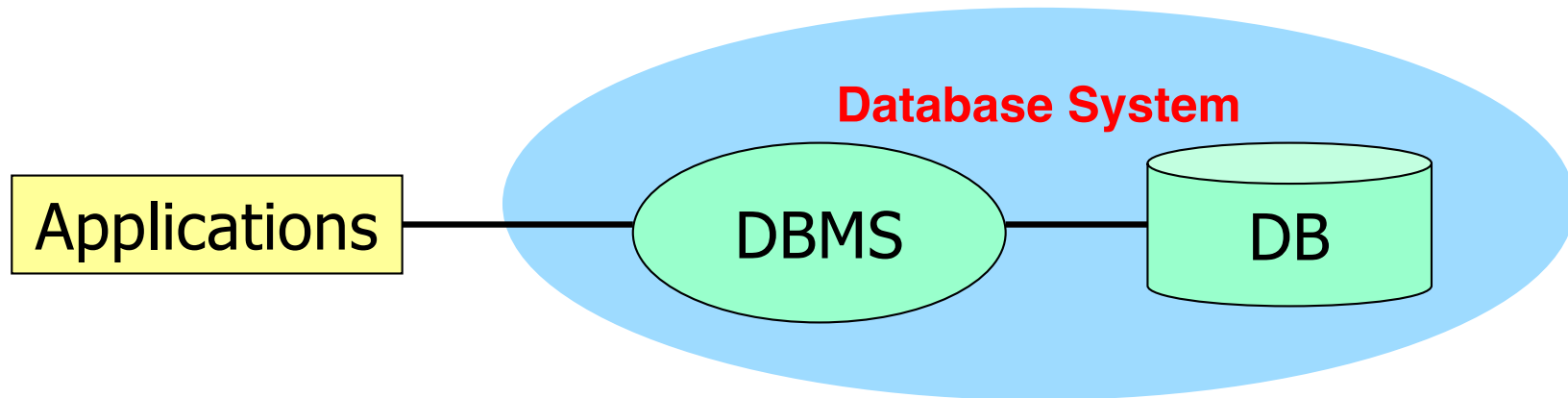
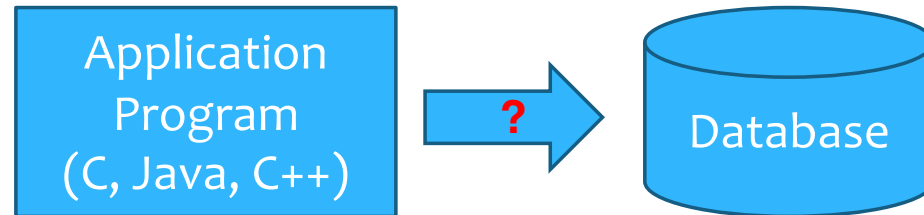
1.1 Database Application Examples

- Enterprise: employees, customers, inventory, orders, supply chain, products, etc.
- Bank: customer, accounts, loans, credit card, banking transactions, etc.
- Universities: students, courses, course registrations, grades, etc.
- Airlines: reservations, schedules, etc.
- Telecommunication: calls, texts, and data usage, etc.
- Online library: articles, patents, published research papers, etc.
- Navigation systems: locations, roads, routes, trains, buses, etc.
- Social-media: users, connections between users, posts made by users, etc.
- Online retailers: goods, order tracking, customized recommendations, etc.

The central aspect of the applications is not a program that processes data, but rather the data themselves!!!



Database System



Database system contains:

Database: Collection of interrelated data about a particular application

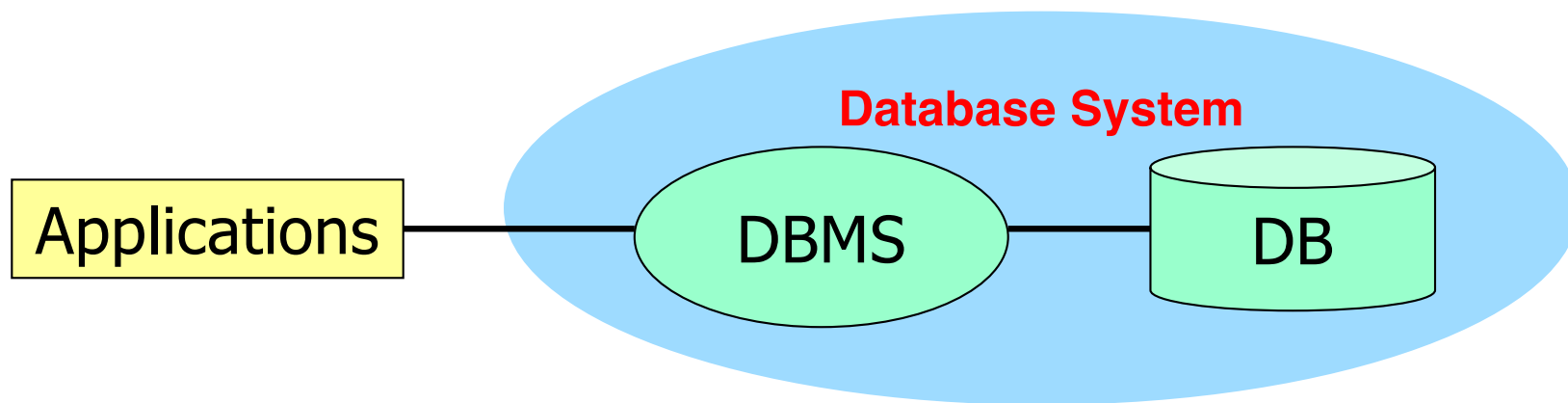
DBMS: Set of programs to access the data

Note: Only the software in the database system is considered here.



Database Management System(DBMS)

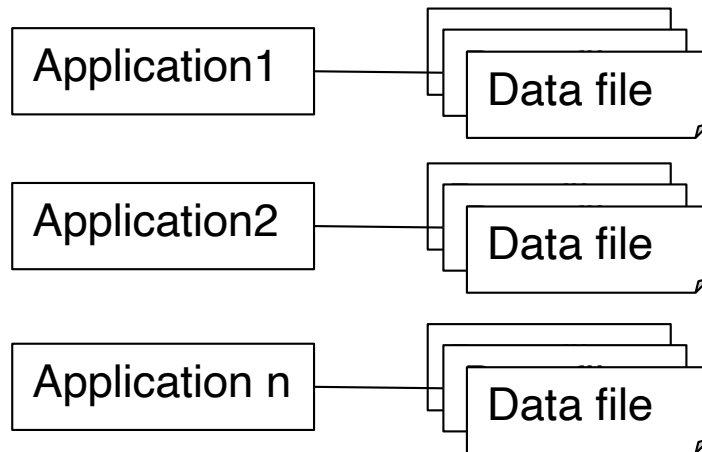
- The primary goal of a **DBMS** is to provide applications with a way to store and retrieve database information that is both convenient and efficient.
- A modern **DBMS** is a complex software system, whose task is to manage a large, complex collection of highly valuable data, which are usually accessed by multiple users and applications at the same time.
- In practice, the terms **database system**, **database**, **database software** and **DBMS (database engine)** are sometimes used synonymously.





1.2 Purpose of Database Systems

In the early days, applications were built directly on top of **file systems**, data was stored in application-oriented files and scattered, which leads to various problems:



■ Data redundancy and inconsistency

Example : staff data in different applications

HR: deptno, staffno, name, deptname, gender, birth, nation, edulevel, position, sal, resume

Teaching: deptno, staffno, name, deptname, gender, age, position, speciality, T-resume

Research: deptno, staffno, name, deptname, gender, age, edulevel, speciality, R-resume

- The same information may be duplicated in several places (files). This redundancy leads to higher storage and access cost. In addition, it may lead to data inconsistency.



Purpose of Database Systems (Cont.)

- **Difficulty in accessing data**
 - Conventional file-processing environments do not allow needed data to be retrieved in a convenient and efficient manner.
- **Data isolation**
 - Retrieving data from other applications is difficult
- **Integrity problems**
 - Integrity constraints (e.g., $18 \leq \text{age} \leq 70$) become “buried” in program code rather than being stated explicitly
 - Hard to add new constraints or change existing ones
- **Atomicity of transactions**
 - A **transaction** is a collection of operations that performs a single logical function in a database application.
 - A transaction should either complete or not happen at all
 - Failures may leave data in an inconsistent state with partial updates carried out

```
read(A)
A := A - 50
write(A)
read(B)
B := B + 50
write(B)
commit
```

Example: Transfer of funds from one account to another



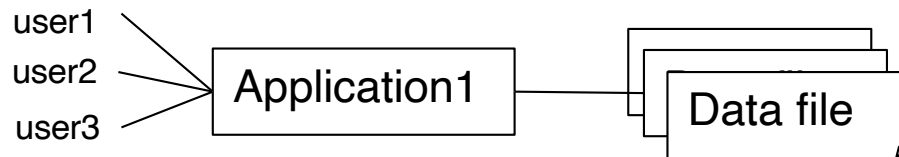
Purpose of Database Systems (Cont.)

■ Concurrent access by multiple users

- Concurrent access needed for performance
- Uncontrolled concurrent accesses can lead to inconsistencies
 - Ex: Two people reading a balance and updating it by withdrawing money at the same time (Fig-1)

■ Security problems

- Hard to provide user access to some, but not all, data



Database systems offer solutions to all the above problems!!!

Fig-1

T_1	T_2
read (A) $A := A - 50$	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B)
write (A) read (B) $B := B + 50$ write (B) commit	 $B := B + temp$ write (B) commit



1.3 Data Models

A DBMS is based on some kind of **data model**.

- DBMS data model is a collection of **tools** for describing
 - Data
 - Data relationships
 - Data semantics
 - Data constraints
- Relational model
- Object-based data models (Object-oriented and Object-relational)
- Semi-structured data model (XML)
- Other older models:
 - Network model
 - Hierarchical model

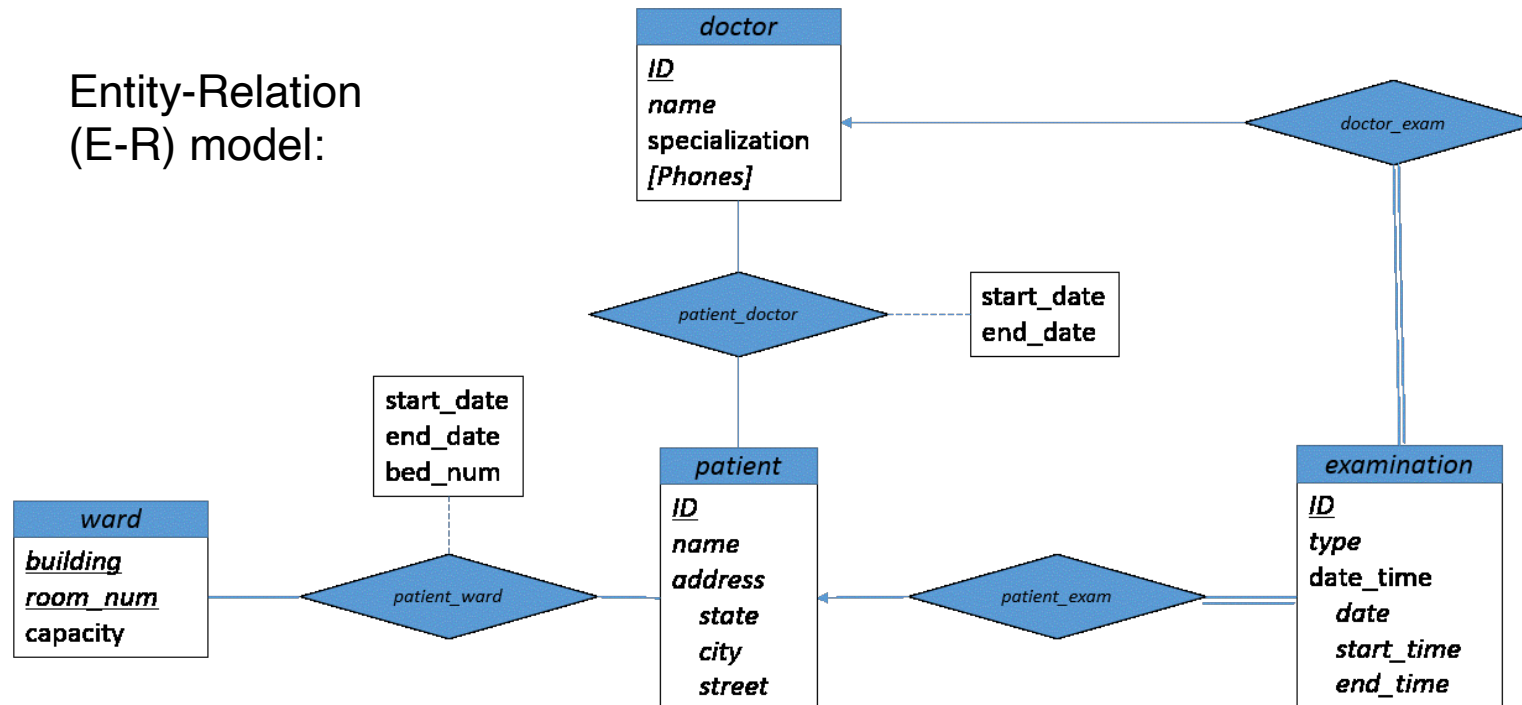


Application Data Models

DBMS data model and **application data model** are two distinct but closely related terms.

- **Application data model:** An abstract formalization of the data objects and relationships found in a particular application domain (e.g. data model of the hospital).
- A set of **concepts** (e.g. attributes, relations, tuples) used in defining such formalizations in DBMS data model.

Entity-Relation
(E-R) model:





1.4 Database Abstraction Levels

Database systems hide the complexity from users through several levels of data abstraction, to simplify users' interactions with the system:

- **Physical level:** about **how** data are **stored**. It is closely related to the system performance. It is **database software dependent** and usually used by database **experts**.
- **Logical level:** about **what** data are stored in database, and the relationships among the data. It's **independent of the** physical level and usually used by database **administrators or developers**.
- **View level:** The **highest** level of abstraction is about only **part** of the entire database. It can **simplify** the data users see and **enhance** the security of the data.

Abstraction is widely used in order to facilitate the research, learning, mastery and development of a complex technical field.



Instances and Schemas

- *We understand a databases at the logical level/physical level through the logical schema/physical schema.*
- **Subschema** – describe different views of the database.
- **Physical schema** – the overall physical structure of the database.
- **Logical schema (database schema)** – the overall logical structure of the database
 - Analogous to **type** information of a variable in a program
- **Instance** – the actual content of the database at a particular point in time
 - Analogous to the **value** of a variable in a program

The physical schema describes the database design at the physical level, while the logical schema describes the database design at the logical level.



Physical Data Independence

- **Physical Data Independence** – the ability to modify the physical schema without changing the logical schema
- Application programs depend on the **logical schema/subschema**
- Application programs are usually of **physical data independence**



1.5 Database Language

- Database Language is a set of languages used to describe, operate, and manage databases. According to their functions and purposes, database languages can be divided into the following types:
 - Data Definition Language
 - Data Manipulation Language

In practice, the data-definition and data-manipulation languages are not two separate languages; instead they simply form parts of a single database language, such as the SQL language.



Data Definition Language (DDL)

- Specification notation for **defining** the database **schema** and related information

Example: **create table** *instructor* (
 ID **char**(5),
 name **varchar**(20),
 dept_name **varchar**(20),
 salary **numeric**(8,2))

- DDL compiler generates a definition stored in a **data dictionary**
- Data dictionary contains **metadata** (i.e., data about data)
 - Database schema
 - Integrity constraints
 - Primary key (ID uniquely identifies instructors)
 - Authorization
 - Who can access what



Data Manipulation Language (DML)

- Language for accessing and updating the data organized by the appropriate data model
 - DML also known as **query** language
- There are basically two types of data-manipulation language
 - **Procedural DML** -- require a user to specify **what** data are needed and **how** to get those data.
 - **Declarative DML** -- require a user to specify **what** data are needed without specifying how to get those data.
- **Declarative** DMLs are usually **easier** to learn and use than are procedural DMLs.
- Declarative DMLs are also referred to as **non-procedural** DMLs



DBMS Access from Application Program

- Non-procedural query languages such as SQL does not support actions such as input from users, output to displays, or communication over the network. Such computations and actions must be written in a **host language**.
- **Application programs** -- written in a **host language**, such as C/C++, Java or Python, and integrated with database languages, such as SQL queries, that access the data in the database.
- Application programs generally access DBMS through one of
 - Language extensions to allow embedded database language
 - Application program interface which allow database language to be sent to a database system.



1.6 Database Design

The process of designing the general structure of the database:

- Logical Design – Deciding on the database schema. Database design requires that we find a “good” database schema. For example:
 - What attributes should we record in the database?
 - What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- Physical Design – Deciding on the physical layout of the database



1.7 Database Engine

- DBMS is partitioned into components that deal with each of the responsibilities of the overall system. These functional components can be broadly divided into:
 - **Storage Manager**
 - **File Manager**: manages the allocation of space on disk storage and the data structures (e.g. data files, data dictionary, indices) used to represent information stored on disk
 - **Buffer Manager**: decides what data to cache in main memory
 - **Query Processor**
 - **DDL Interpreter**: interprets DDL statements and records the definitions in the data dictionary
 - **DML Compiler**: translate DML statements into an optimized evaluation plan consisting of low-level instructions
 - **Query Evaluation Engine**: executes evaluation plans
 - **Transaction Management**
 - **Recovery Manager**: ensures database consistency in case of failure
 - **Concurrency-Control Manager**: controls the interaction among the concurrent transactions to ensure the consistency of the database.



1.8 Database Administrator

A person who has central control over the system is called a **database administrator (DBA)**. Functions of a DBA include:

- Schema definition
- Storage structure and access-method definition
- Schema and physical-organization modification
- Granting of permissions for data access
- Routine maintenance
 - Periodically backing up the database
 - Ensuring that enough free disk space is available for normal operations, and upgrading disk space as required
 - Monitoring jobs running on the database



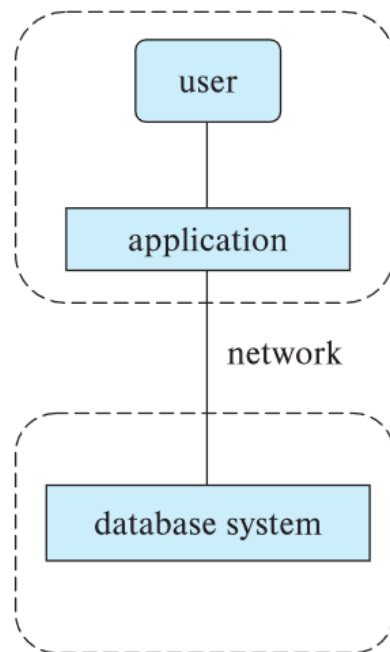
1.9 Database Application System Architecture

Database application systems are usually partitioned into two or three parts:

- **Two-tier architecture** -- the application program resides at the client machine, where it invokes database system functionality at the server machine
- **Three-tier architecture** -- the client machine acts as a front end and does not contain any direct database calls.
 - The client end communicates with an application server, usually through a forms interface.
 - The application server in turn communicates with a database system to access data.

*cheaper
easier to develop
better scalability*

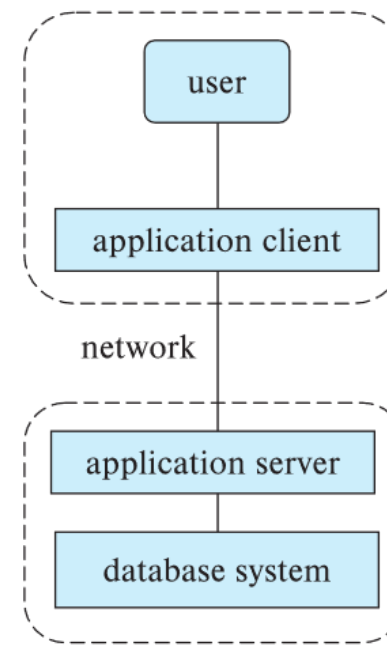
*mainly used for
applications in LAN*



(a) Two-tier architecture

client

server



(b) Three-tier architecture

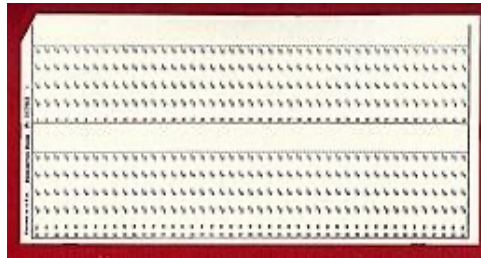
*more secure data
easier to maintain*

*mainly used for
applications on the
Intranet and Internet*



1.9 History of Database Systems

- 1950s and early 1960s:
 - Data processing using magnetic tapes for storage
 - Tapes provided only sequential access
 - Punched cards for input
- Late 1960s and 1970s:
 - Hard disks allowed direct access to data
 - Network and hierarchical data models in widespread use
 - Ted Codd defines the relational data model



半军行数据恢复
RueJunWang@163.com



History of Database Systems (Cont.)

- 1980s:
 - Research in relational prototypes evolve into commercial systems
 - IBM Research System R prototype->SQL/DS
 - UC Berkeley Ingres prototype-> Ingres
 - Oracle releases first commercial relational database
 - SQL becomes industrial standard
 - Relational databases soon dominated the database market
- 1990s:
 - **Large decision support and data-mining applications** -- use data about the past to predict the future and use the predictions to make decisions
 - **Data warehouses** -- gather data from multiple sources at a single site, under a unified schema (which is usually designed to support efficient analysis)
 - Emergence of **Web commerce** (doing business on the Internet)
 - **Parallel and distributed database systems** -- store data across multiple machines, and perform query processing in parallel across multiple machines.
 - **Open-source database systems** (e.g., PostgreSQL, MySQL)
 - **Object-oriented database systems** (commercially unsuccessful)



History of Database Systems (Cont.)

- 2000s
 - “**Non-SQL**” database (e.g., Membase, MongoDB)
 - Big data file systems -- partition and store big data across thousands of computing and storage nodes.
 - **Google File System** (GFS), **Hadoop File System** (HDFS)
 - Big data processing (distributed and parallel)
 - **MapReduce** -- a programming model introduced by Google for processing big data
 - **Hadoop** -- an open-source big data framework developed by Apache for storing and processing big data. Its core is HDFS and MapReduce.
- 2010s
 - **Cloud computing** -- provide low-cost, scalable, on-demand computing service over the Internet. For users, these web services are invisible, just like being blocked by clouds.

Service model of cloud computing:

 - **SaaS** (Software as a Service)
 - **PaaS** (Platform as a Service)
 - **IaaS** (Infrastructure as a Service)



End of Chapter 1